Bevezetés a Rust programozásba

Gembela Gergely

Generikusok és hibakezelés

https://github.com/bmeaut/VIAUBXAV090

Generikusok

- Hasonlóan néznek ki a C++-ban megszokott template-ekhez, de máshogy működnek
  - A szintaxis hasonló (`valami<T>...`, UpperCamelCase névvel)
  - A fordító legyárt minden konkrét behelyettesítés számára egy verziót (Monomorphization)
  - Az általános konstans paraméterek bevezetése folyamatban van
    - Alap integer típusok, `char`, és `bool` paraméterek lehetségesek

Struktúrák és enumok

- Struktúrák/enumok mezőinek típusai lehetnek generikus paraméterek
- Monomorfizáció során a felhasználásoknak megfelelő mennyiségű konkrét verziót generál a fordító
- Életciklus paraméterek is lehetnek generikusak!

```
struct SGeneric<T, U, ...>{...}
struct SGenericLifetime<'a, T, U>{
  f1: &'a T;
  f2: U;
}
```

Függvények és az `impl Trait`

- A `impl Trait` két helyen használható
  - Argumentum típusaként
  - Visszatérési érték típusaként

```
trait T1 {
    fn print_x(&self);
}

fn gen1c(p1: impl T1){
    p1.print_x();
}
```

Függvények és konstansok

- A paraméterek típusai, visszatérési típus, vagy azok paramétere lehet generikus paraméter

```rust
fn FGeneric<T, U, ...>(){...}

fn foo<const N: usize>(arr: [i32; N]) {
    // Used as a type within a function body.
    let x: [i32; N];
    // Used as an expression.
    println!("{}", N * 2);
}
```

Konstansok máshol

```
struct Foo<const N: usize>([i32; N]);
impl<const N: usize> Foo<N> {
    // Used as an associated constant.
    const CONST: usize = N * 4;
}
```

- Nem használható belső argumentum/konstans/statikus/aliased tömb méretéhez
- Literálok, konstans kifejezések (blokk kifejezésekként), és inferált konstansok használhatók
  - a limitációk egy része a parser oldaláról származik

Impl blokkok esetén

- Lehetséges csak részhalmazra implementálni

```
struct Vec<T>{...}
impl Vec<i32>{
  fn only_i32_vec(&self){...}
}
```

- Vagy generikus paramétert bevezetni

```
impl<T> Vec<T>{
  fn something(&self){...}
}
```

Példák

- Nézzünk meg példákat
- Nézzük meg a kibocsátott IR kódot (`cargo rustc -- --emit-llvm`)

Bounds

- Ha generikusokat használunk, szükségünk lehet garanciákra a generikus paraméterekkel kapcsolatban
- C++-ban constraintek és conceptek érik el, Rust-ban *Bound*-ok
- Hatásukra a garantáltan elérhető funkcionalitás használhatóvá válik (pl. `Display` implementálása, vagy garantáltan létező metódusok)
- Üres traitekkel is működik (nézzünk is rá példát (pl. `Copy`))

```
fn must_have_display<T: Display>(){...}
```

Bounds + `where`

- Lehet több megkötés is:

```
fn must_have_display<T: Display + Copy>(){...}
```

- És ugyanez kifejezhető `where`-rel is

```
fn must_have_display<T>() where T: Display + Copy {...}
```

- A `where` clause kifejezőbb lehet mint a fentebbi szintaxis

```
fn must_have_display<T>() where T: Copy, Option<T> : Display {...}
```

Hibakezelés

- Kétféle hibát kezelünk: végzetes, és helyreállítható hibákat
- A végzetes hibák olyan hibák, amelyek oka alapvetően hibásan működő program, ezek *kezelése* a `panic!` makróval történik.
- Van, amikor ez implicit megtörténik (pl. túlindexelés)
- Kiválthatjuk mi magunk is

```
panic!("Ez összeomlott...");
```

- Az alapértelmezett viselkedés ilyenkor *unwind* (a stack bejárása), de az operációs rendszerre is bízhatjuk a takarítást

Hibakezelés

- Ha szép backtrace-t szeretnénk, debug szimbólumokat generálni (tehát cargo … --release) parancsok esetén nem garantált
- Helyreállítható hibák esetén a `Result<T, E> {Ok(T), Err(E)}` enum használatos visszatérési értékként
  - `T` a sikeres, `E` a hibás kimenetel típusa
  - Ezek egészen addig „feljebb adhatók”, amíg valahol tényleg célszerűen kezelhető a hiba

Hibakezelés

- `unwrap()` segítségével pánikká alakítható a hibás alternatíva
- `expect(<message>)` segítségével hibaüzenettel pánikolhatunk
- `unwrap_or()` segítségével kezelhetjük pánik helyett
- A `?` operátor segítségével továbbadhatjuk a hibát a hívónak, vagy kicsomagolhatjuk az eredményt
  - Ebben az esetben fontos lesz a kompatibilitás, a jövő héten bevezetett trait objektumok lesznek a megoldás

Explicit lifetimes recap

- Jellemzően már csak akkor van rá szükség, ha több referencia paraméterünk is van, és nem egyértelmű melyiket adjuk vissza
- Vagy a `'static` lifetime használatakor

```
fn a_or_b<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() > y.len() { x } else { y }
}
```

W5T1: Alakzatok

- Készíts könyvtárat/modult, amely képes legalább 2, 3 és 4 dimenziós vektorok, és 2x2-es, 3x3-as, és 4x4-es mátrixok reprezentálására
- A vektorokra implementáld a skalárszorzat műveletet
- Azonos méretű mátrixok közt valósítsd meg a mátrixszorzást
- Javaslom konstans paraméterek használatát, de nélkülük is megoldható a feladat

Köszönöm a figyelmet!